

GETTING STARTED WITH CONTEXTUAL EMBEDDINGS

Idan Blank
University of California, Los Angeles

Workshop Objectives

- Extract contextual embeddings from a pre-trained LM.
- Get familiar: explore different sentences & different layers.
- Understand some common pitfalls and how to deal with them.
- Apply 1-2 methods for studying embeddings.
- Get reusable code that you can adapt to your own research.

... using **minicons** (by Dr. Kanishka Misra)

... in Google Colab.

Google Colab

- Cloud-hosted platform for Jupyter notebooks.
- Two cell types: executable code & text.
- Execute in order: dependencies within each exercise.
- CPU vs. GPU: Runtime → Change runtime type (re-run all).

Google Colab

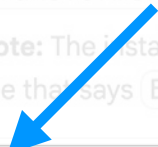
- Cloud-hosted platform for Jupyter notebooks.
- Two cell types: executable code & text.
- Execute in order: dependencies within each exercise.
- CPU vs. GPU: Runtime → Change runtime type (re-run all).
- **Run first code cell** (“Preliminaries”).

Preliminaries: install libraries

We need one library that isn't already in Colab: `minicons`, a toolkit built by Dr. Kanishka Misra on top of the Hugging Face `transformers` library. This package makes it easy to pull a single token's contextual embedding out of a language model.

Note: The install text that appears below (often in orange or red) is normal Python installation chatter, not an error—as long as there's no line that says `ERROR`, you're fine.

✓ 45s



```
# The -q flag just quiets down the installer's normal chatter.  
!pip install minicons -q  
from minicons import cwe
```

Windows: ctrl + enter Mac: cmd + enter

Roadmap

BERT: word representations

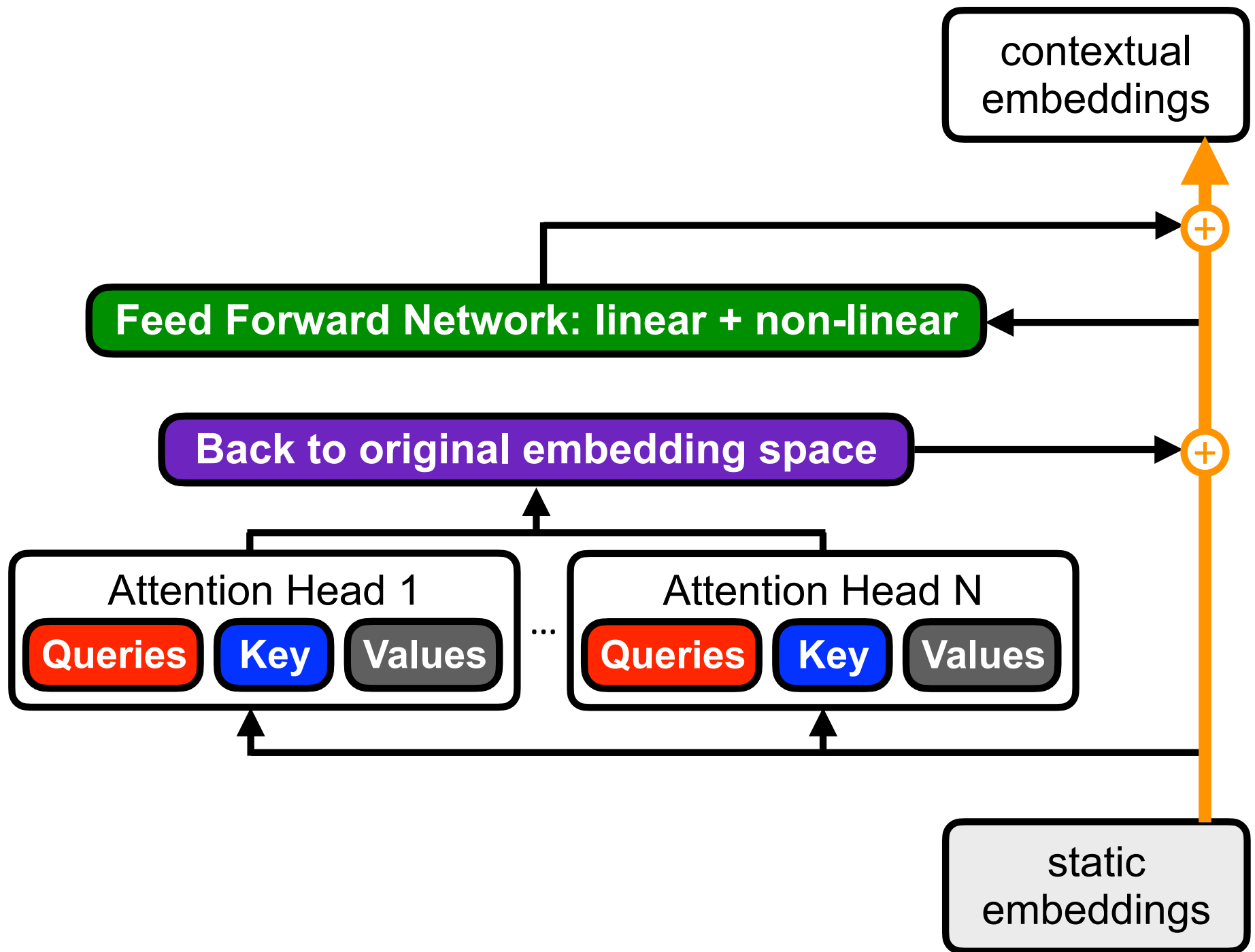
1. Context matters: same word, different sentences
2. Try it yourself

BERT: sentence representations

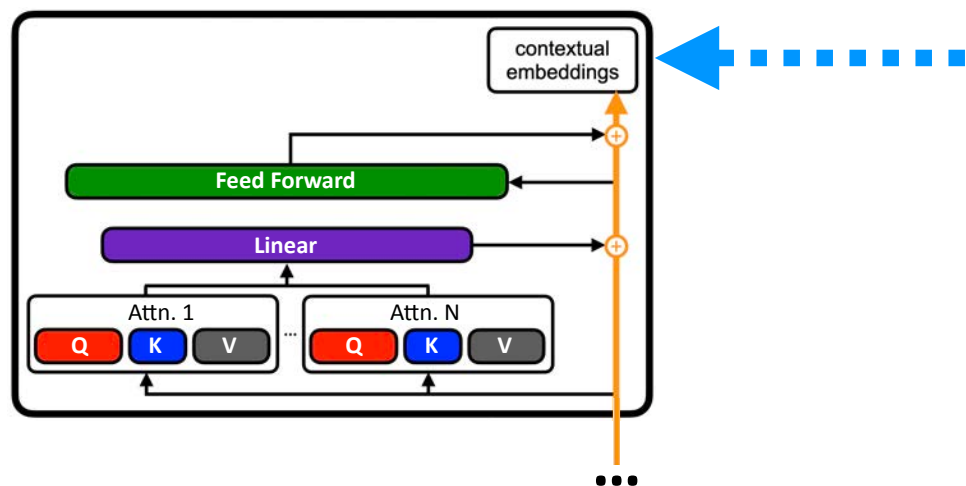
3. Representational similarity: the dative alternation

GPT2-small: sentence representations

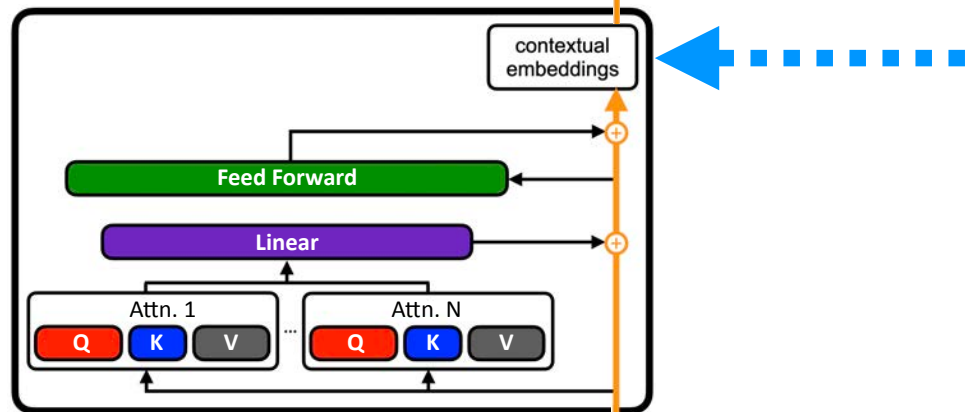
4. Probing: implicit representation of grammaticality?
5. Another probing method (on your own)



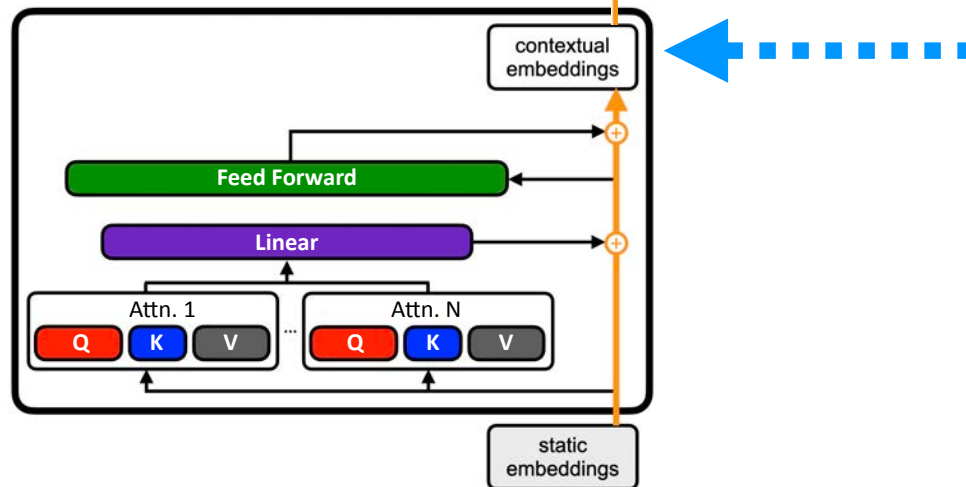
(hidden) layer N



(hidden) layer 2



(hidden) layer 1



Roadmap

BERT: word representations

1. Context matters: same word, different sentences
2. Try it yourself

BERT: sentence representations

3. Representational similarity: the dative alternation

GPT2-small: sentence representations

4. Probing: implicit representation of grammaticality?
5. Another probing method (on your own)

Section 1: One Word, Several Meanings

Running together the following:

1.1. load BERT-base-uncased

- Bidirectional LM
- Loading (almost) all parameters

1.2. Define one sentence with one key word: “bat”

- (sentence, word) pair
- Word must appear exactly as spelled in sentence

1.3. Extract contextual embedding of “bat”

Layer 8 (for now)

Three sentences with “bat”

1. She hit the ball with the **bat** and it flew over the fence.
2. For his birthday the young boy got a wooden **bat**.
3. The **bat** flew out of the cave and found some leftover cake on the ground.

Three embeddings of “bat”.

Which ones should be more/less similar to one another?

Three sentences with “bat”

1. She hit the ball with the **bat** and it flew over the fence.
2. For his birthday the young boy got a wooden **bat**.
3. The **bat** flew out of the cave and found some leftover cake on the ground.

Three embeddings of “bat”.

Which ones should be more/less similar to one another?

Three sentences with “bat”

1. She hit the ball with the **bat** and it flew over the fence.
2. For his birthday the young boy got a wooden **bat**.
3. The **bat** flew out of the cave and found some leftover cake on the ground.

Three embeddings of “bat”.

Which ones should be more/less similar to one another?

Three sentences with “bat”

1. She hit the ball with the **bat** and it flew over the fence.
2. For his birthday the young boy got a wooden **bat**.
3. The **bat** flew out of the cave and found some leftover cake on the ground.

Running together the following:

1.4. Extract 3 contextual embeddings of “bat”

1.5. Examine how BERT tokenizes the sentences

Sub-word tokens (##): averaging the embeddings

1.6. Compute similarities between “bat” embeddings

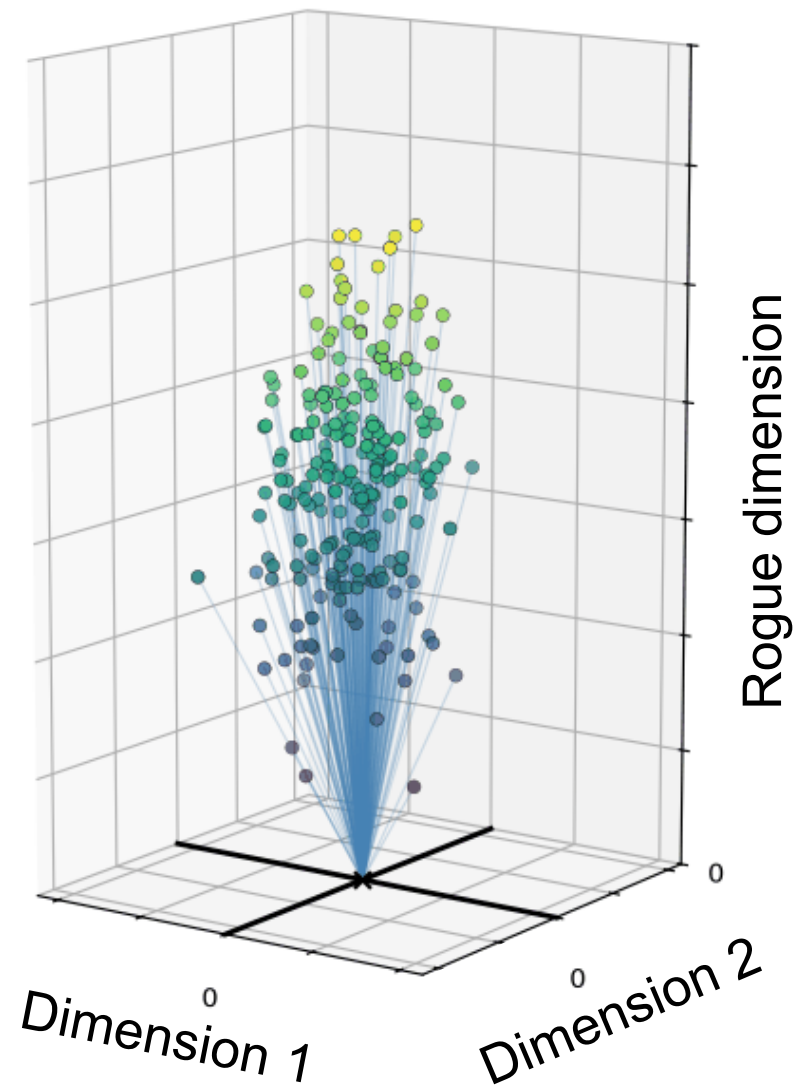
Cosine angle

Rogue Dimensions And Anisotropy

- BERT's embeddings live in a 768-dimensional space.
- Tokens are **not** spread evenly across the space: anisotropy.
 - ◉ Most dimensions: spread around origin, lower variance.
 - ◉ Few rogue dimensions: far from origin, high variance.
- The rogue dimensions dominate cosine similarities.

“cosine similarity [is] a measure over 1-5 rogue dimensions rather than the entire embedding space.

Rogue Dimensions And Anisotropy



Rogue Dimensions And Anisotropy

Solution: **normalize** the value of each unit/dimension.

- Large corpus → feed to LM → get embedding of each token.
- For each unit/dimension d :
 - ◉ Mean activation across tokens (M_d).
 - ◉ Standard deviation of activations across tokens (SD_d).
- For the embeddings of your own stimuli:

$$\text{activation of unit/dimension } d = \frac{\text{raw activation} - M_d}{SD_d}$$

Rogue Dimensions And Anisotropy

Run on your own the following:

1.7. Load means + SDs for each unit (from COCA section)

1.8. Normalize embeddings, recompute similarities

1.9. Compare similarities: raw vs. normalized embeddings

Which Layers Capture Meaning Better?

Run on your own the following:

1.10. Same exercise, different layer

Change LAYER under **# EDIT ME**

1.11. Examining all the layers

1.12. The static embedding layer

Section 2: Your Own Sentences

Run on your own the following:

2.1. Write multiple sentences with the same critical word

- Stimuli + layer under two # EDIT ME sections
- Homonyms / polysemy
- Different construals depending on syntactic structure
- Different contexts emphasize different aspects of meaning

2.2. Extract embeddings, normalize, compute similarities

Roadmap

BERT: word representations

1. Context matters: same word, different sentences
2. Try it yourself

BERT: sentence representations

3. Representational similarity: the dative alternation

GPT2-small: sentence representations

4. Probing: implicit representation of grammaticality?
5. Another probing method (on your own)

Section 3: Representational Geometry

Similarities between the embeddings of **full sentences**.

Running together the following:

3.1. Extracting embeddings for all tokens in a sentence

- [CLS] token: trained to summarize sentence
- [SEP] token: separates sentences

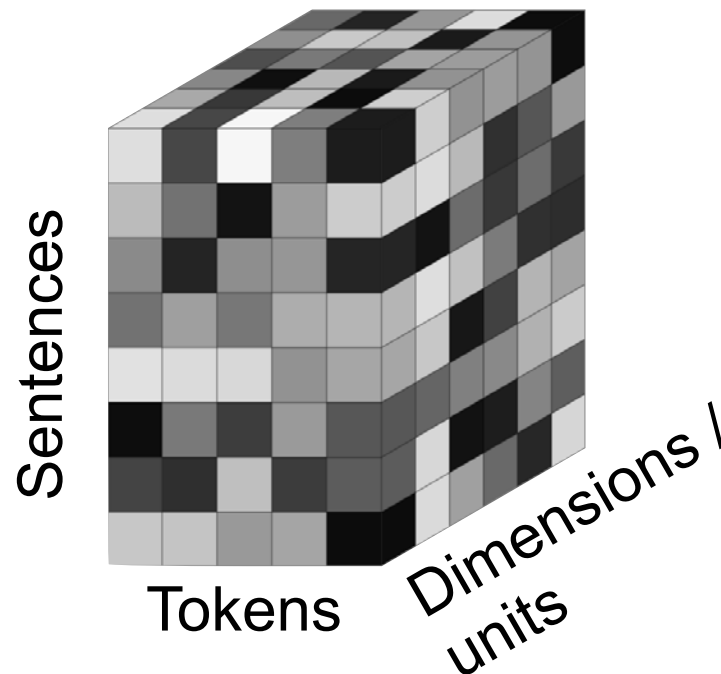
Section 3: Representational Geometry

Similarities between the embeddings of **full sentences**.

Running together the following:

3.2. Helper functions to extract [CLS] from all layers

- `model.encode_text(sentences, layer="all")`
- `hidden_states` is a list with one 3D matrix per layer



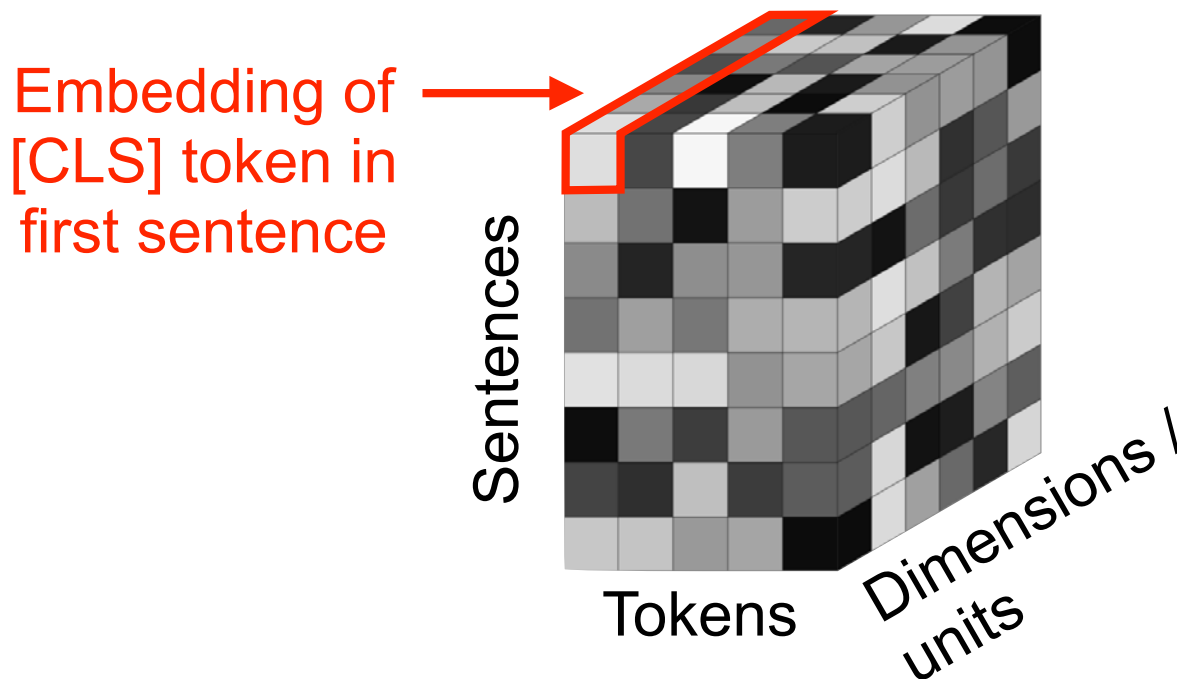
Section 3: Representational Geometry

Similarities between the embeddings of **full sentences**.

Running together the following:

3.2. Helper functions to extract [CLS] from all layers

- `model.encode_text(sentences, layer="all")`
- `hidden_states` is a list with one 3D matrix per layer



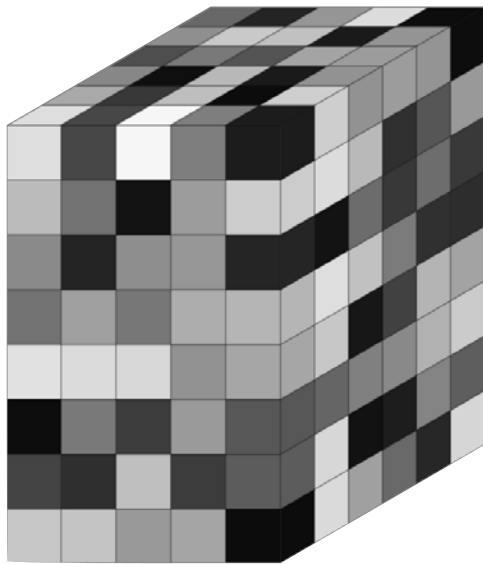
Section 3: Representational Geometry

Similarities between the embeddings of **full sentences**.

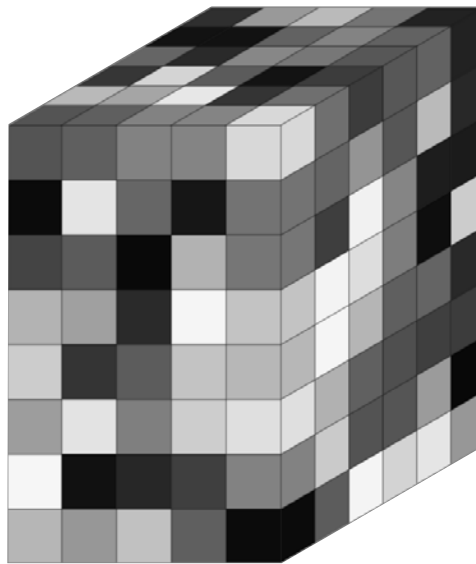
Running together the following:

3.2. Helper functions to extract [CLS] from all layers

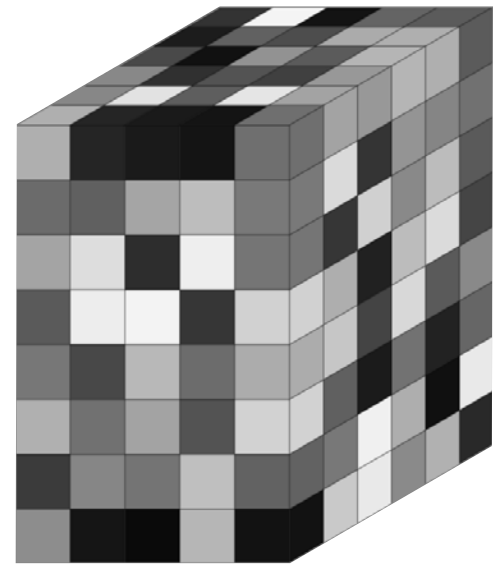
- `model.encode_text(sentences, layer="all")`
- `hidden_states` is a list with one 3D matrix per layer



Layer 1



Layer 2



Layer 3

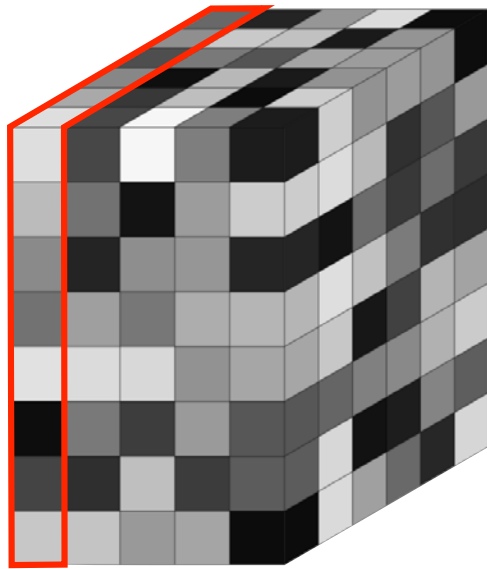
Section 3: Representational Geometry

Similarities between the embeddings of **full sentences**.

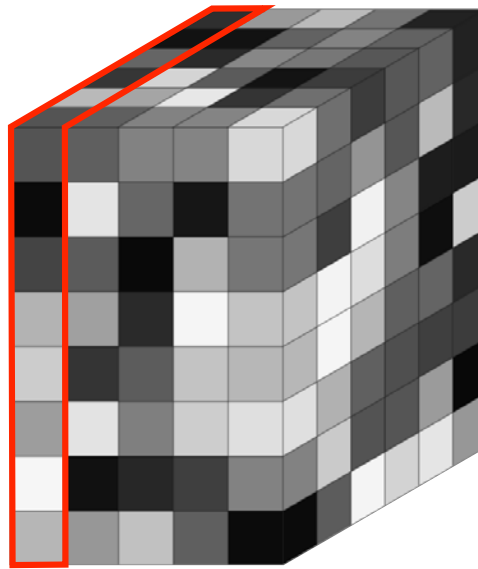
Running together the following:

3.2. Helper functions to extract [CLS] from all layers

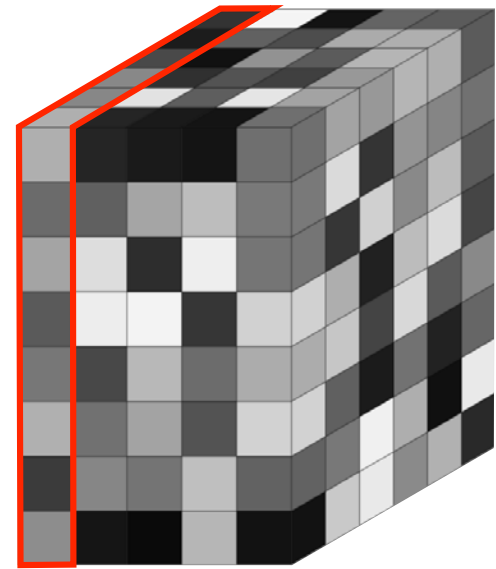
- `model.encode_text(sentences, layer="all")`
- `hidden_states` is a list with one 3D matrix per layer



Layer 1



Layer 2



Layer 3

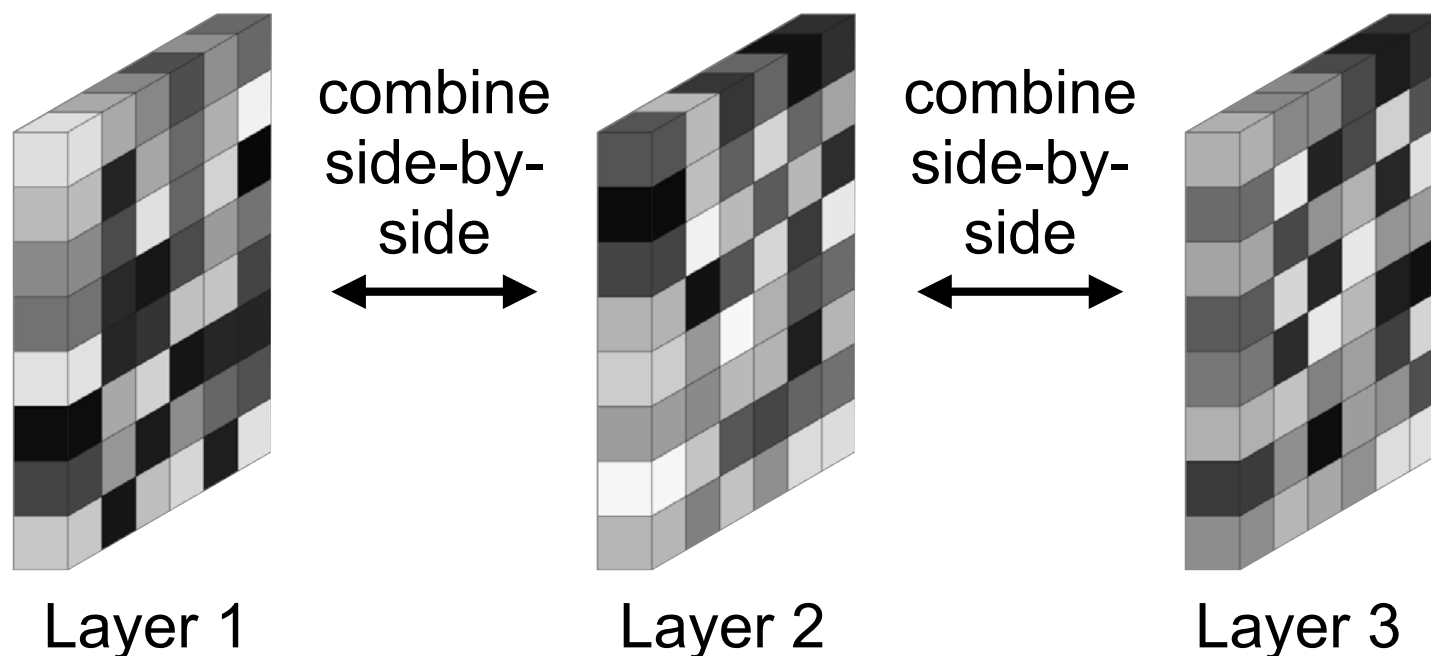
Section 3: Representational Geometry

Similarities between the embeddings of **full sentences**.

Running together the following:

3.2. Helper functions to extract [CLS] from all layers

- `model.encode_text(sentences, layer="all")`
- `hidden_states` is a list with one 3D matrix per layer



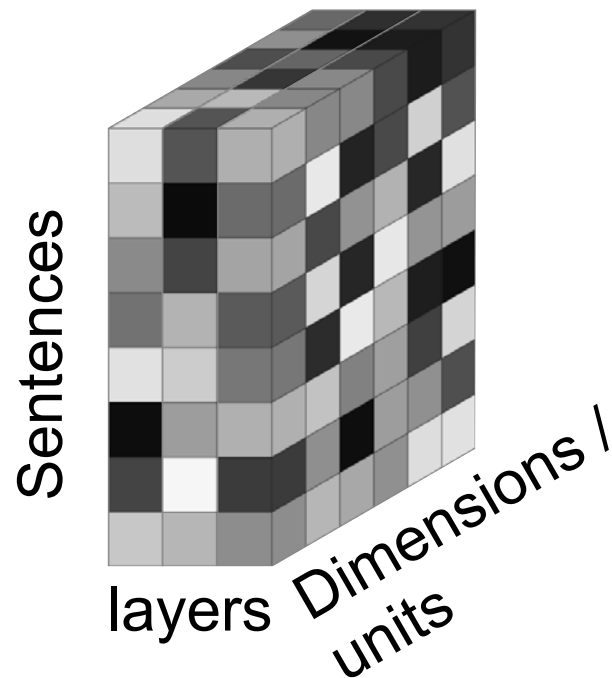
Section 3: Representational Geometry

Similarities between the embeddings of **full sentences**.

Running together the following:

3.2. Helper functions to extract [CLS] from all layers

- `model.encode_text(sentences, layer="all")`
- `hidden_states` is a list with one 3D matrix per layer



Section 3: Representational Geometry

Similarities between the embeddings of **full sentences**.

Running together the following:

3.2. Helper functions to extract [CLS] from all layers

- `model.encode_text(sentences, layer="all")`
- `hidden_states` is a list with one 3D matrix per layer
- `cls_per_layer` is 3D matrix of [CLS] embeddings;
size: [sentences × layers × dimensions]

LM Representational Geometry

The “dative alternation”: two argument-structure constructions, each associating a form with a slightly different meaning.

Direct object (DO): transfer of possession

I gave Sinta the book

They sent the mayor the petition

* They sent the North Pole the petition

Assaf owed her a job

* She brought the table a fork

Prepositional object (PO): change in location

I gave the book to Sinta

They sent the petition to the mayor

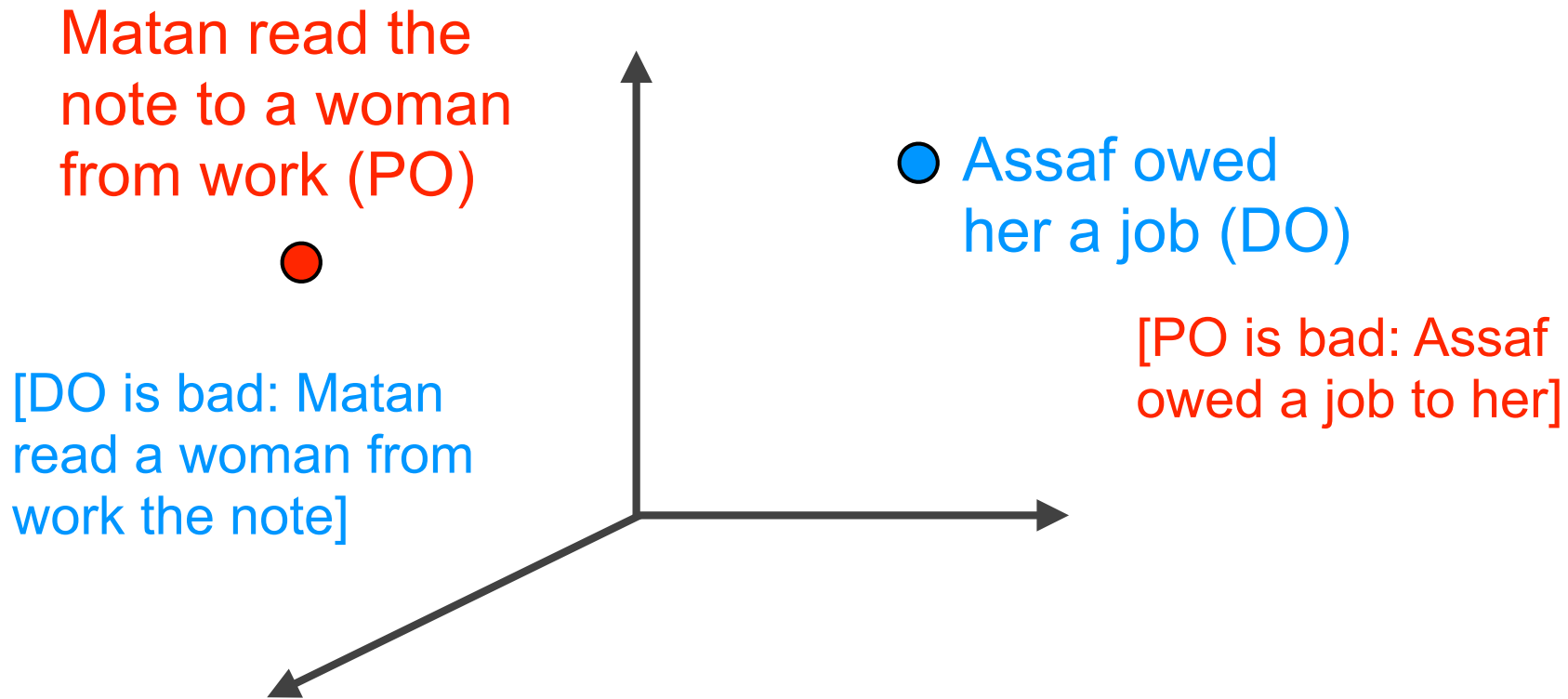
They sent the petition to the North Pole

* Assaf owed a job to her

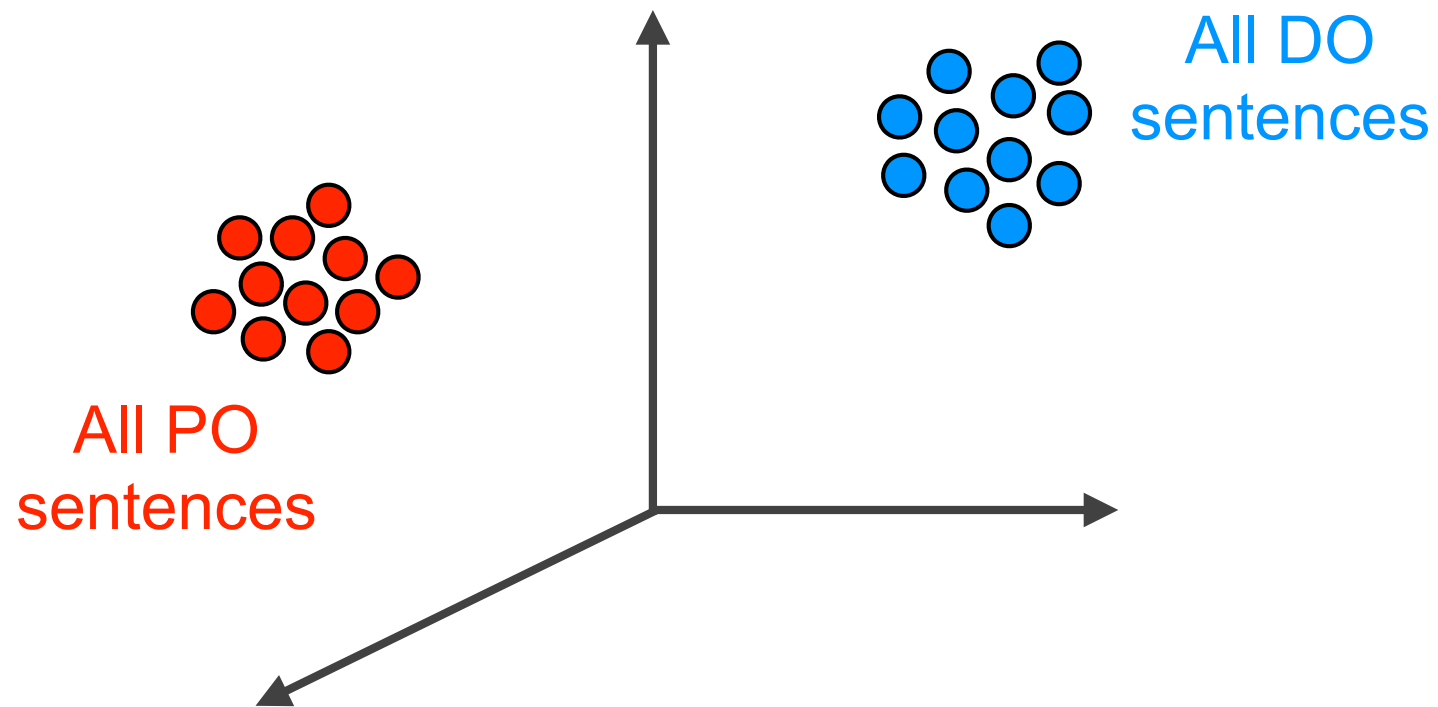
She brought a fork to the table

Is the distinction between **DO** and **PO**
captured in the embeddings of LMs?

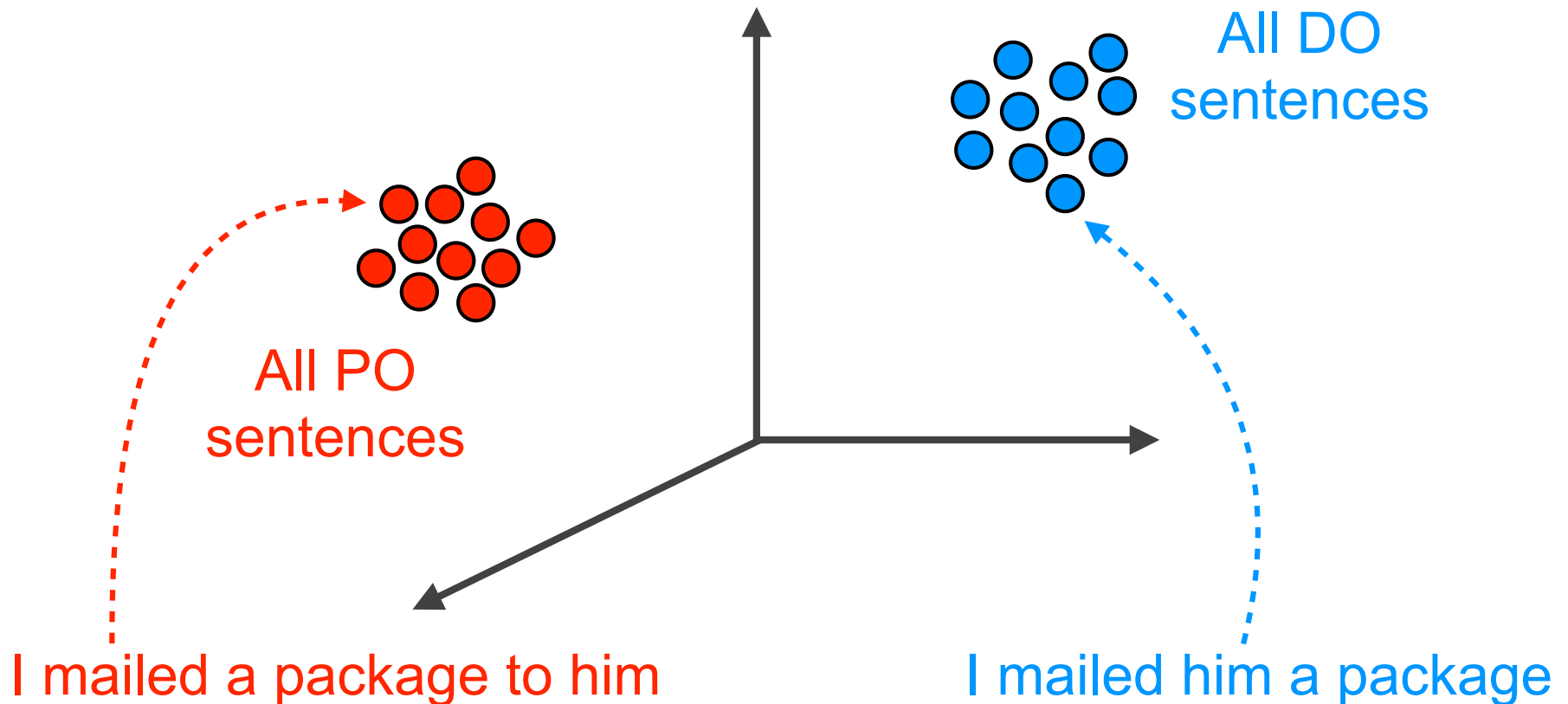
The Dative Alternation in LMs



The Dative Alternation in LMs

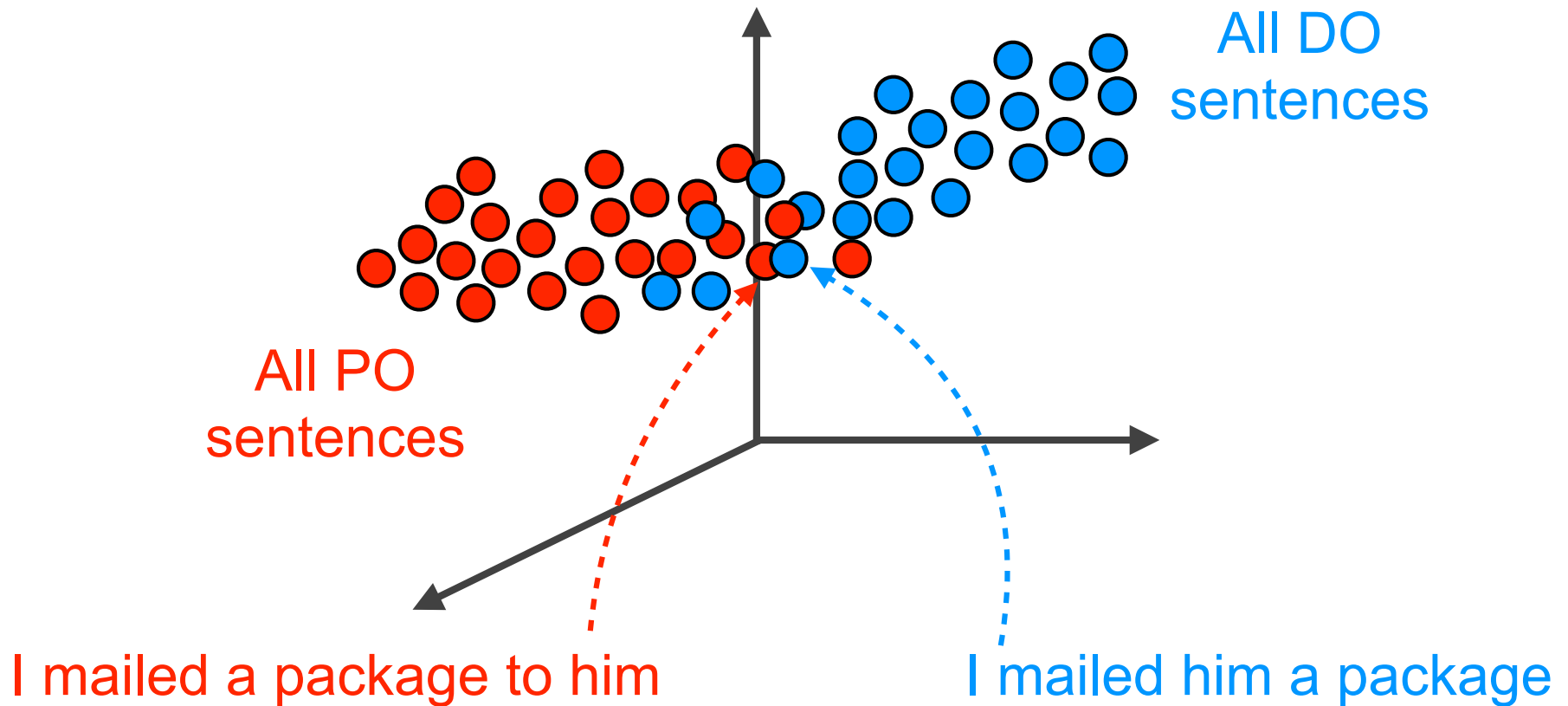


The Dative Alternation in LMs

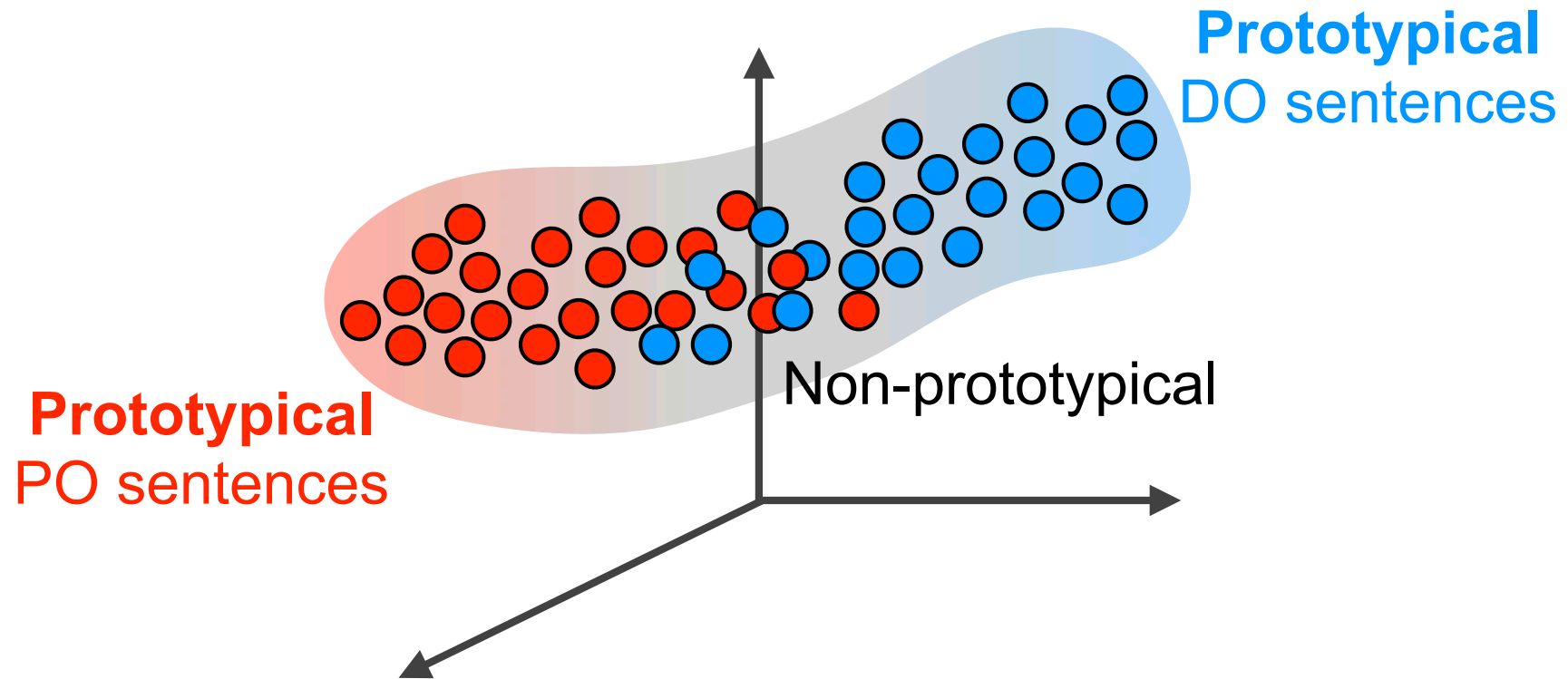


Similar acceptability:
Not a prototypical DO (verb semantics),
not a prototypical PO (information structure)

The Dative Alternation in LMs



The Dative Alternation in LMs



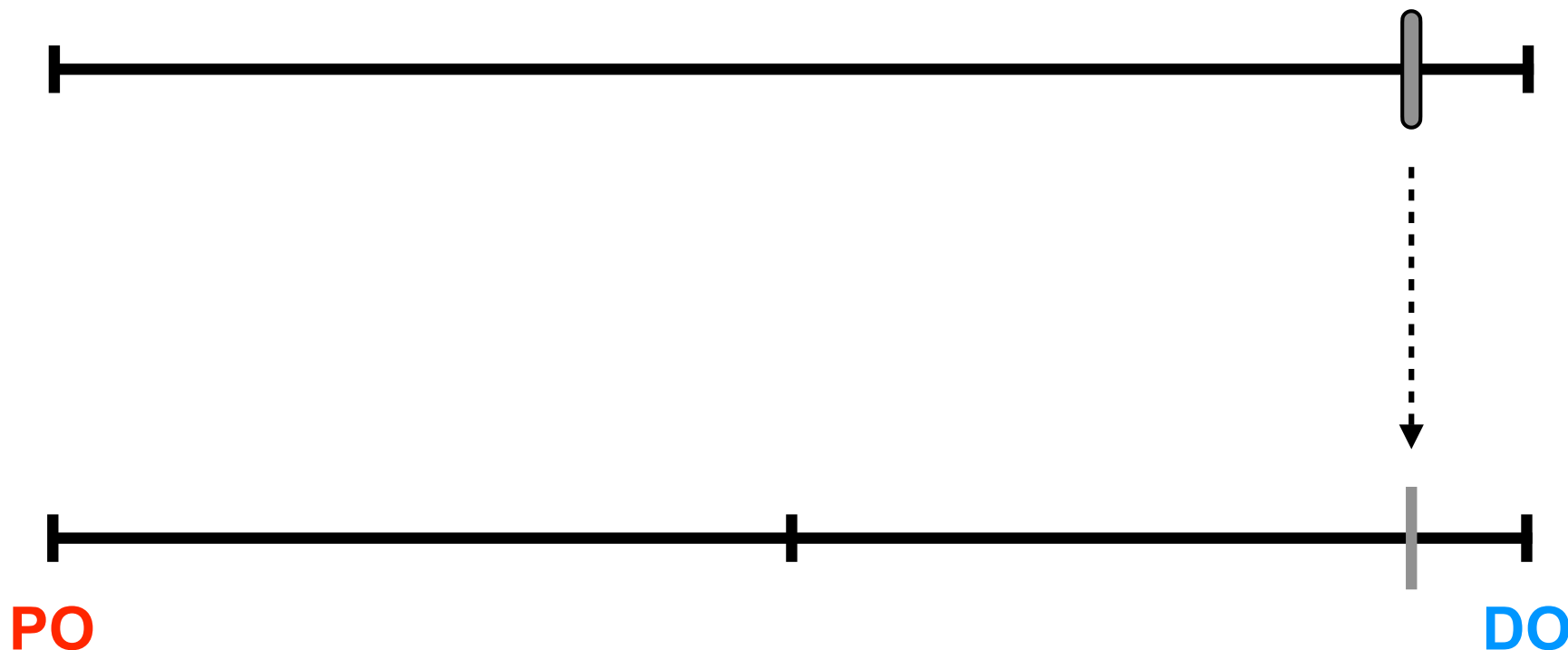
“Meaning-infused grammar”:
Representation of structure organized in a **gradient**
manner, reflecting semantics, information structure, etc.

Meaning-Infused Grammar

- 2500 sentence pairs with human preferences for **PO** or **DO**:

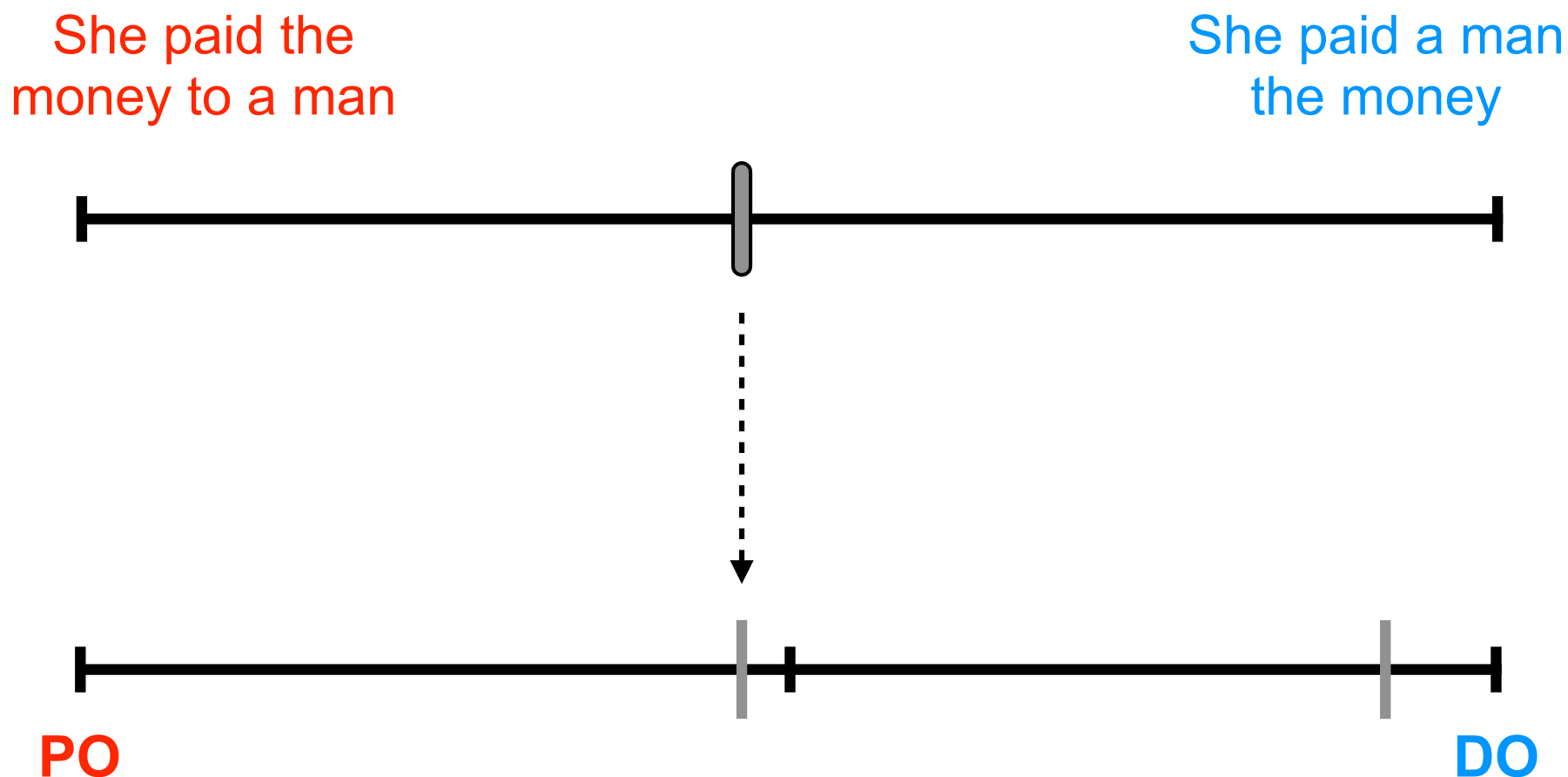
Assaf owed a
job to her

Assaf owed
her a job



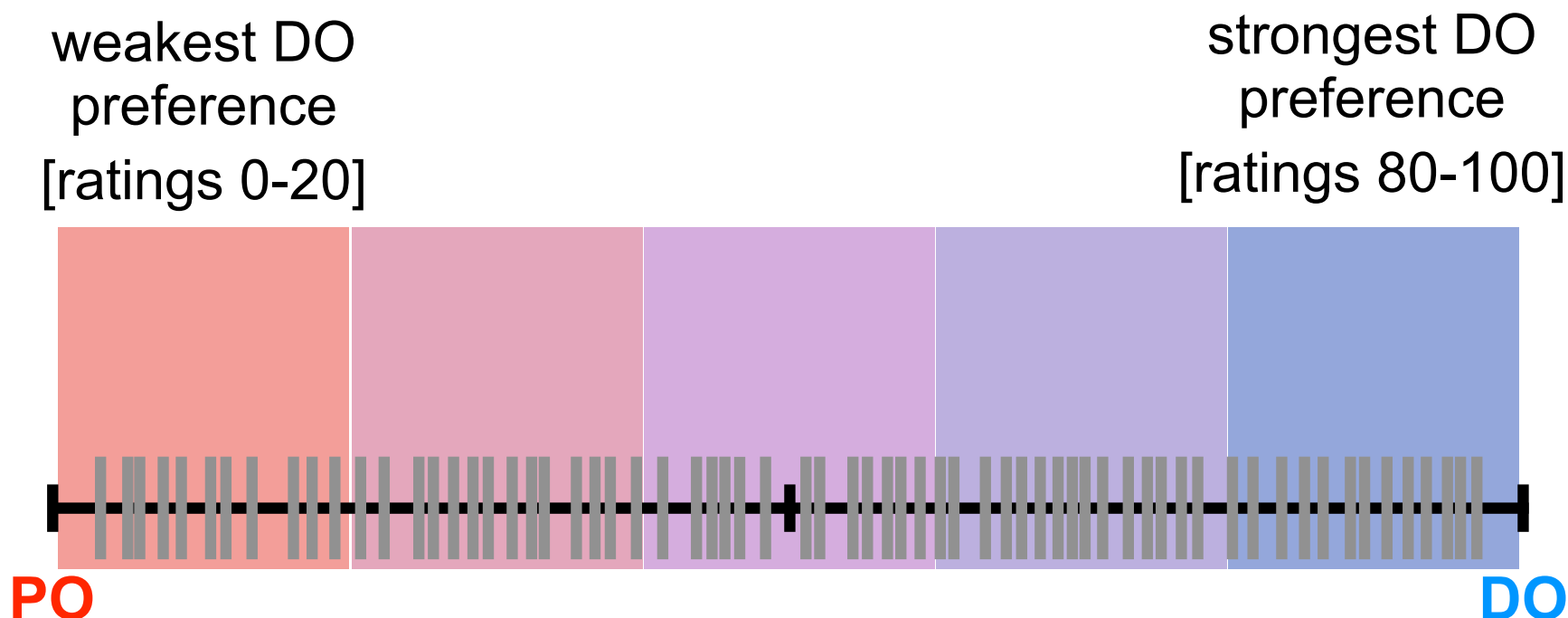
Meaning-Infused Grammar

- 2500 sentence pairs with human preferences for **PO** or **DO**:



Meaning-Infused Grammar

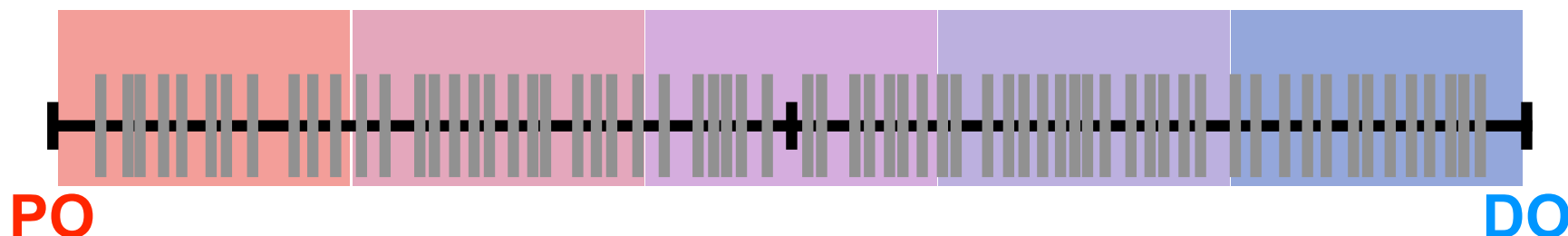
- 2500 sentence pairs with human preferences for **PO** or **DO**:



Meaning-Infused Grammar

- 2500 sentence pairs with human preferences for **PO** or **DO**:
- 5 bins: from weakest to strongest **DO preference**.
From each bin, we will sample 10 pairs closest to the center.
- Feed each sentence to LM, extract contextual embedding from some layer n .
- In each bin of preference, calculate **similarity** between each **DO** sentence and its **PO** counterpart.

What should we expect?



The Dative Alternation in LMs

Run on your own the following:

3.3. Load behavioral data

3.4. Sample sentence pairs per bin

Inspect example sentence pairs.

3.5. Extract [CLS] embeddings

- From each layer
- For each sentence in each sentence pair

The Dative Alternation in LMs

Run on your own the following:

3.6. Load statistics for normalizing embeddings

Identical to 1.7.

3.7. Helper function for computing cosine similarities

3.8. Main analysis

- Choose layer under **# EDIT ME**
- Inspect plot at the bottom of the output
- Re-run with different layers

3.9. Interpret the plot

Roadmap

BERT: word representations

1. Context matters: same word, different sentences
2. Try it yourself

BERT: sentence representations

3. Representational similarity: the dative alternation

GPT2-small: sentence representations

4. Probing: implicit representation of grammaticality?
5. Another probing method (on your own)

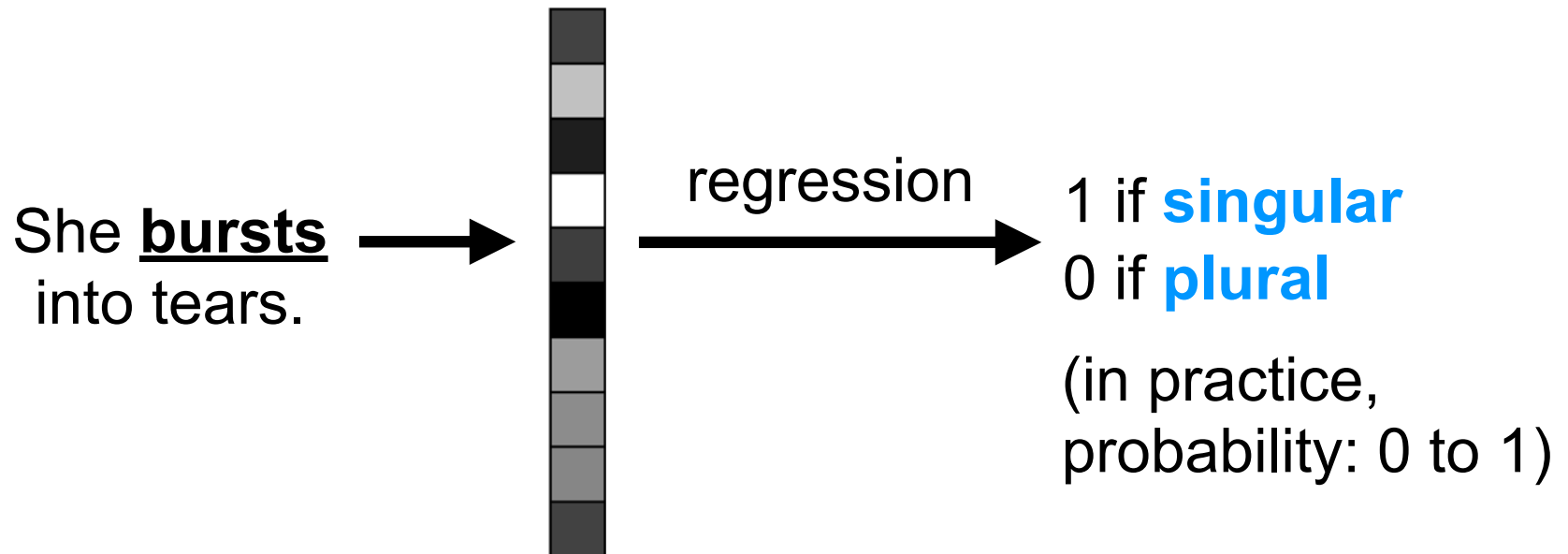
A single embedding packs a lot of information.
Overall similarities between embeddings reflect
the most dominant information.

What other information is in there,
hidden in different subspaces?

Section 4: A Probing Classifier

Direct way of asking: is information about linguistic feature **F** encoded in the embeddings?

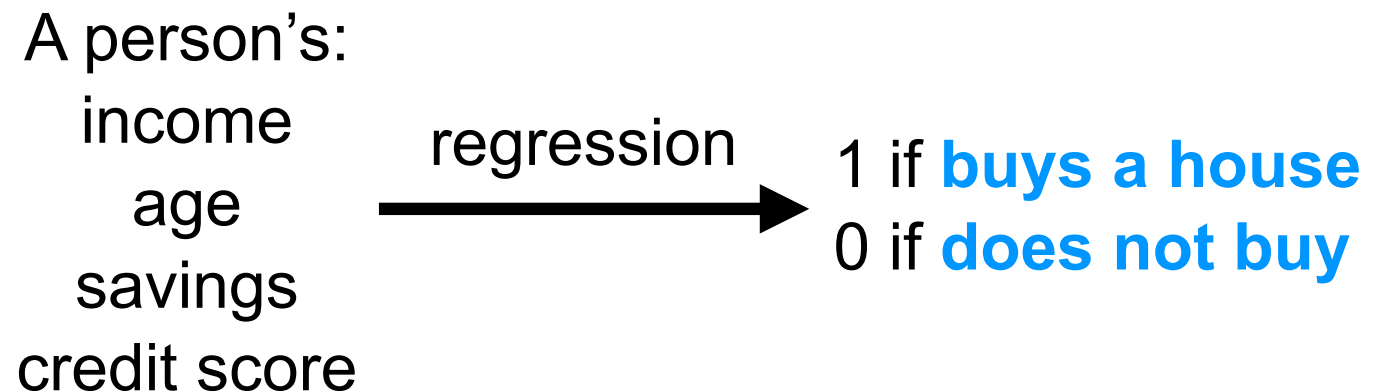
- **Probing classifier**: small statistical model that we train, takes an embedding and predicts the value of **F**.
 - If we take the embedding of a verb, can a classifier predict whether that verb is **singular or plural**?



Section 4: A Probing Classifier

Direct way of asking: is information about linguistic feature ***F*** encoded in the embeddings?

- **Probing classifier**: small statistical model that we train, takes an embedding and predicts the value of ***F***.
 - ◉ If we take the embedding of a verb, can a classifier predict whether that verb is **singular or plural**?



Section 4: A Probing Classifier

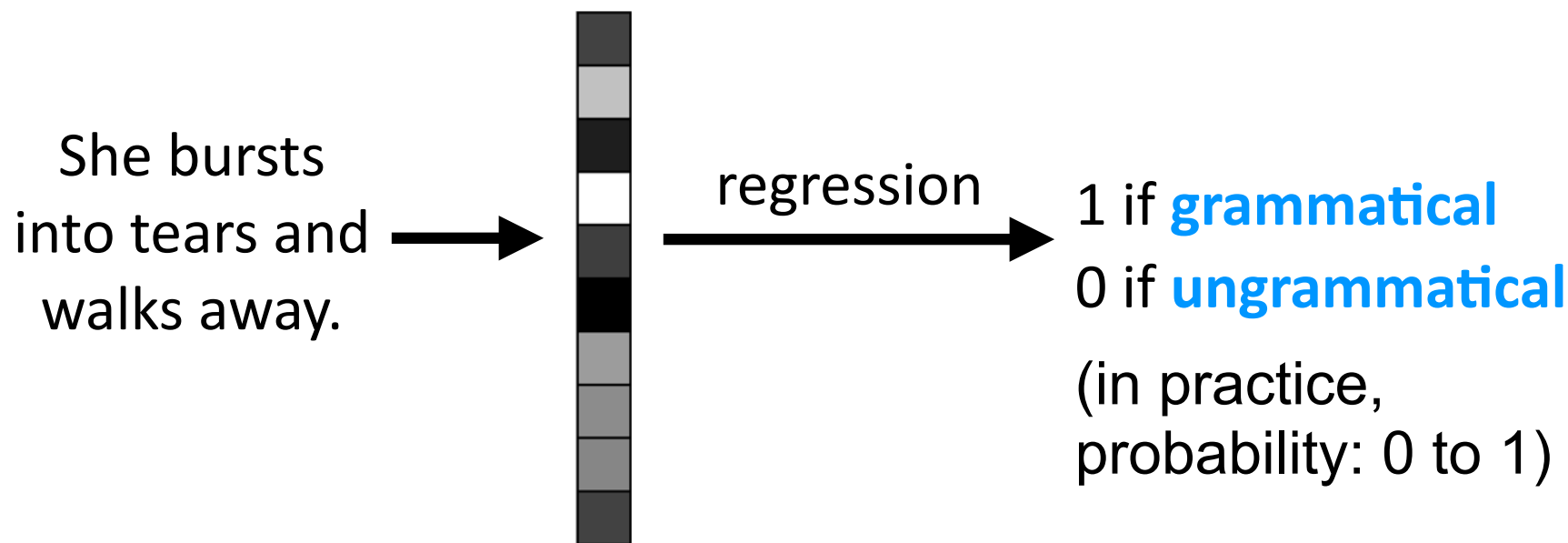
Direct way of asking: is information about linguistic feature F encoded in the embeddings?

- **Probing classifier:** small statistical model that we train, takes an embedding and predicts the value of F .
 - If we take the embedding of a verb, can a classifier predict whether that verb is **singular or plural**?
 - If we take the embedding of a verb's direct object, can a classifier predict how **plausible** that direct object is? (e.g., for “eat”: “cake” > “sand” > “democracy”).
 - If we take the embedding of a noun, can a classifier predict whether it's the **direct object or indirect object of a verb**?
- Classifier succeeds in predicting $F \rightarrow$ LM embeddings implicitly encode F (or some approximation of it).

A Probe For Grammaticality

If we take the embedding of a sentence, can a classifier predict whether that sentence is **grammatical or not**?

- Use an embedding that represents an entire sentence.
- Our probe: logistic regression.



A Probe For Grammaticality

If we take the embedding of a sentence, can a classifier predict whether that sentence is **grammatical or not**?

- Use an embedding that represents an entire sentence.
- Our probe: logistic regression from embedding (X) to **label** (y).
Test whether grammaticality is encoded in a **linear** manner.
- First, **train** the probe on example pairs [embedding, label].
- Then, **test** whether the probe has learned to predict grammaticality: embeddings of new, unseen sentences.

Training The Probing Classifier

A “corpus” of real vs. corrupted sentences:

Insertion	She bursts into tears and walks away.	She bursts into tears fringe and walks away.
Deletion	What is his usual food , Nelly?	What his, Nelly?
Shuffling	Now we shall have some discussion , we'll see to that.	Now have discussion shall we some , we'll see to that.

A Different LM: GPT2-small

- Uni-directional LM: no future looking.
- No [CLS] token.
- Can represent entire sentence with embedding of **last token**:
 - ◉ Only one that can attend to all other tokens in the sentence.
 - ◉ Often punctuation: “.” or “?”
- Can also average the embeddings of all tokens in the sentence (available in the notebook, try on your own).

Setting Up The Analysis

Running together the following:

4.1. Load GPT2-small

4.2. Load mini-corpus of real vs. corrupted sentences

Examine some examples

4.3. Examine how GPT2 tokenizes the sentences

Sub-word tokens: **Ġ** marks beginnings (not continuations).

4.4. Helper function to subsample the corpus

1,200 sentences (600 pairs) instead of 6,000, to save time

4.5. Helper function to extract sentence embeddings

Before we continue: Run **section 4.6**
(extract embedding for the 1,200 sentences)

Choose LAYER under **# EDIT ME**

Do **not** change POOLING_METHOD

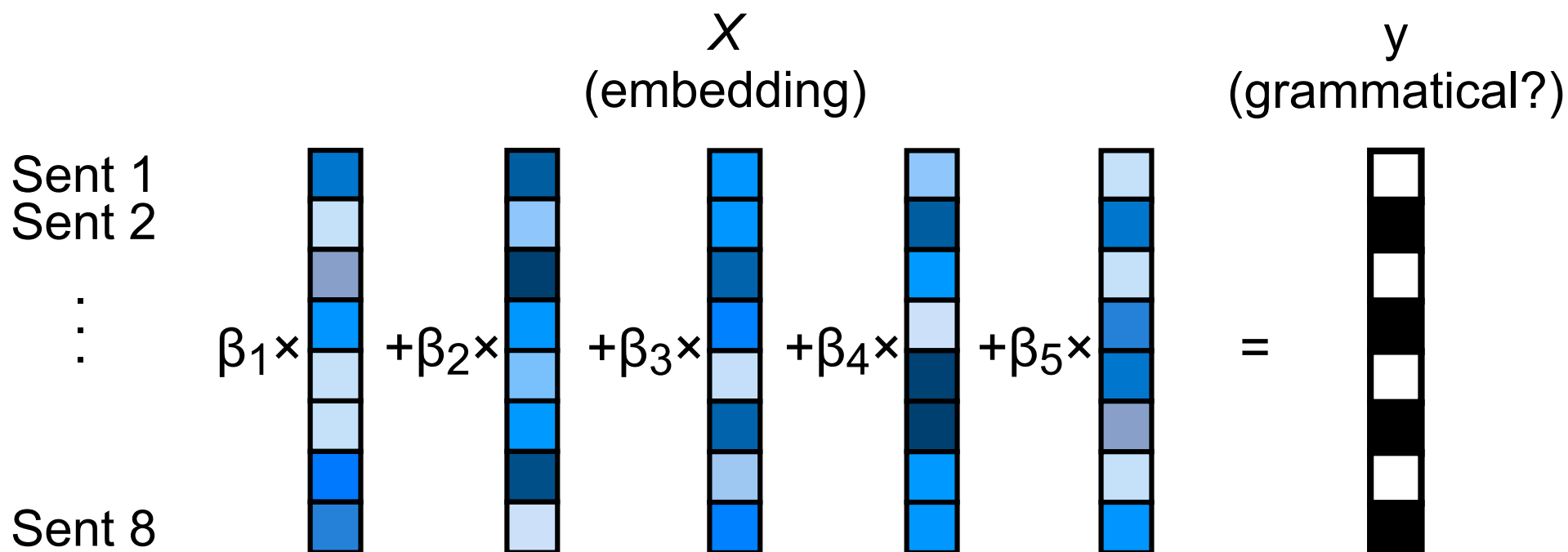
Under The Hood: Padding Issues

- When sending several sentences together to LM (batch):
 - ◉ All sentences “padded” to have same number of tokens.
 - ◉ Example: longest sentence is 20 tokens → a sentence with 14 tokens will get 6 copies of a “padding token” at the end.
- **Danger 1: contaminated representations**
 - ◉ Bidirectional LMs only: real tokens attend to future padding.
 - ◉ `encode_text` in minicons prevents this automatically.
- **Danger 2: grabbing embeddings of wrong token(s)**
 - ◉ `encode_text` returns null embedding for each padding token.
 - ◉ Last token might not be a real token: you’ll get `[0, 0, ..., 0]`.
 - ◉ Averaging all embeddings might include null embeddings.
 - ◉ Notebook deals with this issue in section 4.5.

Training A Logistic Regression Classifier

	X (embedding)	y (grammatical?)
Sent 1		
Sent 2		
$\vdots$		
$\vdots$		
$\vdots$		
$\vdots$		
$\vdots$		
Sent 8		

Training A Logistic Regression Classifier



- Each embedding has 768 dimensions $\rightarrow$ 768 β weights (+1)
- With only 1,200 sentences, the classifier might overfit.
- Regularization: penalty on high weights.
- Parameter C : lower C = stronger penalty.

Choosing The Regularization Parameter

- Choose a value of C to try.
- Train classifier on 80% of the corpus (training subset).
Evaluate accuracy on remaining 20% (development subset):
 - ◉ Grammatical sentence: classifier correct if probability ≥ 0.5
 - ◉ Ungrammatical sentence: classifier correct if probability < 0.5
- Repeat for other values of C .
Choose the C that gives the best accuracy.
- Re-train the classifier on all sentences, with chosen C .

Training The Probing Classifier

Run on your own the following:

4.7. Normalize embeddings (because anisotropy)

Now the M s and SD s for normalization are for GPT2

4.8. Helper functions for logistic regression

4.9. Training the logistic regression (probing classifier)

4.10. Sanity check accuracy on development subset

After Training, Time For Testing!

Did the classifier find a robust representation of grammaticality inside the embeddings?

- Stress test: instead of random corruptions, use violations of specific grammatical regularities.
- BLiMP: benchmark of human curated minimal pairs, targeting specific types of grammatical knowledge (agreement, gaps...).

Grammatical:

Raymond is selling
this sketch.

Ungrammatical:

Raymond is selling
this sketches.

After Training, Time For Testing!

Did the classifier find a robust representation of grammaticality inside the embeddings?

- Stress test: instead of random corruptions, use violations of specific grammatical regularities.
- BLiMP: benchmark of human curated minimal pairs, targeting specific types of grammatical knowledge (agreement, gaps...).
- Stronger test of grammaticality: less “glaring” violations.
- Lenient evaluation: for each minimal pair, does classifier give higher probability to grammatical sentence?
(doesn't matter if probabilities are higher or lower than 0.5).

Testing The Probing Classifier

Run on your own the following:

4.11. Load minimal pairs from BLiMP

- Only 300 sentences (6 grammatical regularities)
- Inspect example minimal pairs

4.12. Extract and normalize sentence embeddings

4.13. Evaluate probing classifier on the minimal pairs

Inspect predictions for example pairs, and overall accuracy

4.14. Plot summary of results

4.15. Question

Using Other LMs

Minicons should wrap any LM on Hugging Face:

- `model = cwe.CWE("gpt2", device=device)`
- `model = cwe.CWE("EleutherAI/pythia-410m", device=device)`
- `model = cwe.CWE("meta-llama/Llama-2-7b-hf", device=device)`

Remember:

- Bidirectional vs. unidirectional LMs.
- Normalization stats in the notebook are only for BERT + GPT2.
- LM size: might need to use GPU (or run on cluster).
- Parameter count might be different than what you expect.
- Access requests for some LMs (see “preliminaries” code).